

TIME\_WAIT 和 CLOSE\_WAIT 是 TCP 连接关闭过程中的两种状态，分别出现在主动关闭连接和被动关闭连接的一方。它们的存在是为了确保 TCP 连接的可靠关闭，但在高并发场景下，过多的 TIME\_WAIT 或 CLOSE\_WAIT 状态可能导致系统资源耗尽，影响性能。以下是它们的原因及规避方法。

---

## 一、TIME\_WAIT 状态

### 1. \*\*TIME\_WAIT 的原因\*\*

- TIME\_WAIT

是 TCP 连接关闭的最后一个状态，出现在\*\*主动关闭连接的一方\*\*。

- 当一方调用

close() 或 shutdown() 主动关闭连接时，会进入 TIME\_WAIT 状态。

- \*\*作用\*\*：

- a. 确保最后一个 ACK 报文能够到达对方。
- b. 防止旧连接的延迟报文干扰新连接。

- \*\*持续时间\*\*：

- TIME\_WAIT 状态的持续时间通常是 \*\*2MSL (Maximum Segment Lifetime) \*\*。
- 在 Linux 中，MSL 默认为 60 秒，因此 TIME\_WAIT 状态持续 120 秒。

### 2. \*\*TIME\_WAIT 的影响\*\*

- 在高并发场景下，大量 TIME\_WAIT 状态会占用系统资源（如端口和内存），导致新连接无法建立。
- 常见于短连接频繁创建和关闭的场景（如 HTTP 请求）。

### 3. \*\*规避 TIME\_WAIT 的方法\*\*

- \*\*启用 TCP 重用\*\*：

- 修改内核参数，允许重用处于 TIME\_WAIT 状态的连接：

```
1 echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse
2
```

- \*\*启用 TCP 快速回收\*\*：

- 修改内核参数，加快 TIME\_WAIT 状态的回收：

```
1 echo 1 > /proc/sys/net/ipv4/tcp_tw_recycle
2
```

- **\*\*注意\*\***：在 NAT 环境下，启用 tcp\_tw\_recycle 可能导致连接问题，需谨慎使用。
- **\*\*使用长连接\*\***：
  - 避免频繁创建和关闭短连接，改用长连接（如 HTTP Keep-Alive）。
- **\*\*调整 TIME\_WAIT 超时时间\*\***：
  - 修改内核参数，减少 TIME\_WAIT 状态的持续时间：

```
1 echo 30 > /proc/sys/net/ipv4/tcp_fin_timeout
2
```

---

## 二、CLOSE\_WAIT 状态

### 1. **\*\*CLOSE\_WAIT 的原因\*\***

- CLOSE\_WAIT 是 TCP 连接关闭的中间状态，出现在**\*\*被动关闭连接的一方\*\***。
- 当一方收到对方的 FIN 报文后，会进入 CLOSE\_WAIT 状态，等待应用程序调用 close() 关闭连接。
- **\*\*常见原因\*\***：
  - 应用程序未正确关闭连接（如未调用 close()）。
  - 应用程序处理连接过慢，导致连接长时间处于 CLOSE\_WAIT 状态。

### 2. **\*\*CLOSE\_WAIT 的影响\*\***

- 大量 CLOSE\_WAIT 状态会占用系统资源（如文件描述符），导致无法建立新连接。
- 常见于服务器端，尤其是后端服务未正确处理连接关闭的场景。

### 3. **\*\*规避 CLOSE\_WAIT 的方法\*\***

- **\*\*检查代码逻辑\*\***：
  - 确保应用程序在收到 FIN 报文后，正确调用 close() 关闭连接。
- **\*\*设置超时时间\*\***：
  - 为连接设置超时时间，避免连接长时间未关闭：

```
1 conn.SetDeadline(time.Now().Add(time.Second * 30))
2
```

- **使用连接池**：
  - 在高并发场景下，使用连接池管理连接，避免频繁创建和关闭连接。
- **监控和报警**：
  - 监控 CLOSE\_WAIT 状态的数量，及时发现和处理问题。

### 三、TIME\_WAIT 和 CLOSE\_WAIT 的对比

| 状态         | 出现位置      | 原因                  | 持续时间           | 规避方法                   |
|------------|-----------|---------------------|----------------|------------------------|
| TIME_WAIT  | 主动关闭连接的一方 | 确保连接可靠关闭，防止旧报文干扰新连接 | 2MSL（通常 120 秒） | 启用 TCP 重用、使用长连接、调整超时时间 |
| CLOSE_WAIT | 被动关闭连接的一方 | 应用程序未正确关闭连接         | 直到调用 close()   | 检查代码逻辑、设置超时时间、使用连接池    |

### 四、总结

- **TIME\_WAIT** 是主动关闭连接的一方进入的状态，确保连接可靠关闭。可以通过启用 TCP 重用、使用长连接等方式规避。
- **CLOSE\_WAIT** 是被动关闭连接的一方进入的状态，通常由应用程序未正确关闭连接引起。需要检查代码逻辑并设置超时时间。
- 通过合理配置和优化，可以有效减少 TIME\_WAIT 和 CLOSE\_WAIT 状态对系统性能的影响。